

```

# %% [markdown]
# Homework sessions 1-3, part A
# Data folder: environment variable MILK10K_DIR, or ../milk10k next to this folder.

# %%
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))

import os

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import roc_auc_score

from milk10k import (
    FIGURE_DIR,
    IMG_DIR,
    OUTPUT_DIR,
    RARE_DX,
    SEED,
    LesionDataset,
    aggregate_predictions,
    answer_five_questions,
    assert_labels,
    assert_split_properties,
    build_lesions,
    circular_distance,
    compare_split_strategies,
    dummy_costs,
    group_hue_value,
    jitter_changes,
    jpeg_roundtrip,
    leakage_experiment,
    load_tables,
    memory_bytes,
    numpy_geometry,
    pillow_geometry,
    split_lesions,
    with_dx,
)

from torchvision import transforms

FIGURE_DIR.mkdir(parents=True, exist_ok=True)
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
meta, gt = load_tables()
print("data folder:", IMG_DIR.parent)
print("images:", len(meta), "lesions in gt:", len(gt))

# %% [markdown]
# ## A1.1 Cross-check the two label files

# %%
violations, mapping = assert_labels(meta, gt)
print(mapping)
multi = mapping.index[(mapping > 0).sum(axis=1) != 1]
print("classes with more than one diagnosis_1:")
print(mapping.loc[multi] if len(multi) else "none")

# %% [markdown]
# AKIEC is the only one of the 11 classes that does not map to a single diagnosis_1.
# In the lesion table it is split between Indeterminate and Malignant, so an 11-class
# prediction cannot be converted into diagnosis_1 for that class. The other ten classes
# each sit inside one diagnosis_1 value, and diagnosis_3 sits inside one diagnosis_2,
# which sits inside one diagnosis_1. The two images of a lesion agree on age, sex, site,
# and the diagnosis fields, and every lesion has exactly one positive gt column.
# On a new medical dataset I would always repeat the id match, the one-hot check, and
# the check that repeated images of the same case share one label. A broken one-hot row
# or two images of one lesion with different diagnoses is the most dangerous failure:
# the model is then trained against a target that contradicts itself.

```

```

# %% [markdown]
# ## A1.2 Five questions

# %%
answers = answer_five_questions(meta, gt)
for key, value in answers.items():
    print(f"{key}: {value}")

# %% [markdown]
# 1. The count above is the number of BCC lesions with age at least 70 and site head/neck.
# 2. The class named in q2_class has the highest share of altered images; q2_altered_pct is that share.
# 3. Youngest and oldest MEL ages and their gap are q3_youngest, q3_oldest, and q3_gap.
# 4. A missing site is informative: the class in q4_class moves by q4_points percentage points between lesions with a site and lesions without one. Missingness is not a random blank.
# 5. The malignant percentage is much higher when confirmation is histopathology than when it is a single clinical assessment. The archive was filled mostly with lesions that were already sent for biopsy, so it is not a sample of everyone who has a spot on the skin.

# %% [markdown]
# ## A1.3 Lesion-level table

# %%
lesions = build_lesions(meta, gt)
lesion_path = OUTPUT_DIR / "lesions.csv"
lesions.to_csv(lesion_path, index=False)
print(lesions.head())
print("saved", lesion_path, "rows", len(lesions))

# %% [markdown]
# ## A1.4 Task framing

# %%
SPEC = """
The model receives one skin-lesion photograph, either dermoscopic or a clinical close-up, and may also see age, sex, and body site. It outputs benign, malignant, or indeterminate. A clinician would see that output when deciding whether to biopsy or to send the patient home, before a pathology report exists. Optimise malignant recall, or a cost that charges a missed cancer much more than an extra clinic visit. Accuracy is the wrong target because most study images are already malignant. MILK10k is not the clinic in two ways. Lesions were kept because they were biopsied or otherwise selected for a study, so cancer is far more common than in screening. The photos are two framed study images per lesion, not one casual picture from a new patient.
""".strip()
print(SPEC)
print("words:", len(SPEC.split()))
assert len(SPEC.split()) <= 200

# %% [markdown]
# ## A2.1 Arrays, views, and memory

# %%
sample_id = meta.iloc[0]["isic_id"]
from PIL import Image

original = np.array(Image.open(IMG_DIR / f"{sample_id}.jpg").convert("RGB"))
ours = numpy_geometry(original)
theirs = pillow_geometry(original)
for name in ours:
    assert np.array_equal(np.ascontiguousarray(ours[name]), theirs[name]), name
    print(f"{name}: shares memory with the original? {np.shares_memory(original, ours[name])}")

base = original.copy()
view = base[:, :-1]
before = int(base[0, -1, 0])
view[0, 0, 0] = (before + 1) % 256
print("view write changed the original pixel:", int(base[0, -1, 0]) != before)
copied = base[:, :-1].copy()
print("copy shares memory?", np.shares_memory(base, copied))

widths, heights = [], []

```

```

for isic_id in meta["isic_id"]:
    with Image.open(IMG_DIR / f"{isic_id}.jpg") as image:
        width, height = image.size
        widths.append(width)
        heights.append(height)
uint8_original = sum(memory_bytes(h, w, 1, 1) for h, w in zip(heights, widths))
float_original = uint8_original * 4
n_images = len(meta)
uint8_224 = memory_bytes(224, 224, n_images, 1)
float_224 = memory_bytes(224, 224, n_images, 4)
for label, nbytes in {
    "uint8 original": uint8_original,
    "float32 original": float_original,
    "uint8 224": uint8_224,
    "float32 224": float_224,
}.items():
    print(f"{label}: {nbytes / 1024 ** 3:.2f} GB")

# %% [markdown]
# Left-right, up-down, rot90, and transpose are views: they share memory with the array.
# Writing into the left-right view changed the matching pixel of the original. A .copy() does
not.
# Keeping every original image as float32 needs more RAM than a typical laptop should spend on
# the images alone, because the operating system and the model also need memory. At 224x224 the
# float32 copy is smaller but still large. Do not pre-load the set. Read a batch from disk,
# which is what the loader in part B does.

# %% [markdown]
# ## A2.2 JPEG compression and file size

# %%
rows = []
decoded = {}
for quality in (95, 75, 50, 25, 10):
    size, image, score = jpeg_roundtrip(original, quality)
    rows.append({"quality": quality, "bytes": size, "psnr": round(score, 2)})
    decoded[quality] = image
print(pd.DataFrame(rows))
height, width = original.shape[:2]
y0, x0 = height // 2 - 40, width // 2 - 40
best = max(rows, key=lambda row: row["psnr"])
worst = min(rows, key=lambda row: row["psnr"])
fig, axes = plt.subplots(1, 2, figsize=(6, 3))
axes[0].imshow(decoded[best["quality"]][y0 : y0 + 80, x0 : x0 + 80])
axes[0].set_title(f"best PSNR, quality {best['quality']}")
axes[1].imshow(decoded[worst["quality"]][y0 : y0 + 80, x0 : x0 + 80])
axes[1].set_title(f"worst PSNR, quality {worst['quality']}")
for axis in axes:
    axis.axis("off")
fig.savefig(FIGURE_DIR / "jpeg_crops.png", bbox_inches="tight")
plt.close()

meta = meta.copy()
meta["bytes"] = [
    os.path.getsize(IMG_DIR / f"{isic_id}.jpg") for isic_id in meta["isic_id"]
]
pair = meta[meta["diagnosis_1"].isin(["Benign", "Malignant"])]
fig, axis = plt.subplots(figsize=(6, 3))
axis.boxplot(
    [
        pair.loc[pair["diagnosis_1"] == "Benign", "bytes"] / 1024,
        pair.loc[pair["diagnosis_1"] == "Malignant", "bytes"] / 1024,
    ],
    tick_labels=["Benign", "Malignant"],
)
axis.set_ylabel("File size (KB)")
fig.savefig(FIGURE_DIR / "file_size_boxplot.png", bbox_inches="tight")
plt.close()
for image_type in ("dermoscopic", "clinical: close-up"):
    subset = pair[pair["image_type"] == image_type]
    score = roc_auc_score(subset["diagnosis_1"].eq("Malignant"), subset["bytes"])
    print(f"ROC-AUC of file size, {image_type}: {score:.3f}")

```

```

# %% [markdown]
# The images are already JPEG, so the pixels contain compression blocks. Saving them
# again is a second compression. Quality 75 can land closer to this file than quality 95,
# because 95 is not the quality the archive used. Resize the decoded pixels, not the file.
# File size is not a useful malignancy score: ROC-AUC is about 0.48 for dermoscopic images
# and 0.51 for clinical close-ups, which is chance. Do not use file size as an input.

# %% [markdown]
# ## A3.1 Does a leaked split change the metric?

# %%
images = meta.merge(with_dx(gt)[["lesion_id", "dx"]], on="lesion_id")
leak = leakage_experiment(images, seeds=[0, 1, 2, 3, 4])
for name, result in leak.items():
    print(
        f"{name}: RF balanced accuracy {result['rf_mean']:.3f} ± {result['rf_std']:.3f}; "
        f"oracle {result['oracle_balanced_accuracy']:.3f}; "
        f"sibling in train {result['pct_sibling_in_train']:.1f}%"
    )

# %% [markdown]
# The random forest sees only age, sex, site, and image type. Those fields are already the
# same on both photos of a lesion, except image type. Putting one photo in train and the
# other in test therefore gives this model almost no new information, so its balanced
# accuracy barely moves. The sibling oracle is the check that matters for a model that can
# recognise the picture itself: when the other view is in train, the oracle copies that
# label, and a large share of a naive test set has that sibling available. A CNN could do
# the same by matching the lesion rather than learning the disease. "The forest did not
# improve" is not a reason to split by image. The leak is harmless only for features that
# are already shared. It is not harmless for the pixels.

# %% [markdown]
# ## A3.2 Three-way lesion split

# %%
rare_hits = assert_split_properties(
    lesions, val_size=0.2, test_size=0.2, seeds=list(range(10))
)
print("seeds in which the rare class is in both val and test:")
print(rare_hits)
print("lesions per rare class:")
print(lesions["dx"].value_counts().reindex(RARE_DX))

# %% [markdown]
# A class can be present in val and in test and still be too small to score. MAL_OTH has
# only nine lesions, so a 20% test set holds one or two of them. One mistake moves recall
# from 0 to 1. Presence in the split is not a stable estimate of that class.

# %% [markdown]
# ## A3.3 Three folding strategies

# %%
comparison = compare_split_strategies(images, seed=SEED)
print(comparison.to_string(index=False))

# %% [markdown]
# GroupKFold keeps a lesion on one side but does not keep class rates steady.
# StratifiedKFold keeps class rates but places the two photos of one lesion on opposite
# sides. StratifiedGroupKFold is the only strategy that does both, so it is the one to use.

# %% [markdown]
# ## A3.4 Metrics and the cost of a mistake

# %%
train_ids, val_ids, test_ids = split_lesions(lesions, 0.2, 0.2, SEED)
test_mask = images["lesion_id"].isin(test_ids)
train_mask = images["lesion_id"].isin(train_ids)
train_idx = np.flatnonzero(train_mask.to_numpy())
test_idx = np.flatnonzero(test_mask.to_numpy())
cost_table, matrix = dummy_costs(images, train_idx, test_idx, SEED)
print(cost_table.to_string(index=False))

```

```

print("confusion matrix, stratified random, rows true Benign / Indeterminate / Malignant")
print(matrix)
by_accuracy = cost_table.sort_values("accuracy", ascending=False)["predictor"].tolist()
by_cost = cost_table.sort_values("expected_cost")["predictor"].tolist()
print("rank by accuracy:", by_accuracy)
print("rank by expected cost (low is better):", by_cost)

# %% [markdown]
# Missing a malignant lesion costs 50, an unnecessary check of a benign lesion costs 1,
# and any indeterminate mistake costs 5. Always calling malignant wins on accuracy because
# most images are malignant, and it also wins on this cost matrix because the missed-cancer
# penalty dominates. The ranking agrees here, but only because the cost of a miss is large
# and the majority class is the dangerous one. If the clinic were mostly benign, accuracy
# would still reward the majority class and the cost would not.

# %% [markdown]
# ## A3.5 Is the colour augmentation larger than the class gap?

# %%
benign_ids = images.loc[images["diagnosis_1"].eq("Benign"), "isic_id"].sample(100,
random_state=SEED)
malignant_ids = images.loc[images["diagnosis_1"].eq("Malignant"), "isic_id"].sample(100,
random_state=SEED)
benign_paths = [IMG_DIR / f"{isic_id}.jpg" for isic_id in benign_ids]
malignant_paths = [IMG_DIR / f"{isic_id}.jpg" for isic_id in malignant_ids]
benign_hue, benign_v = group_hue_value(benign_paths)
malignant_hue, malignant_v = group_hue_value(malignant_paths)
hue_gap = circular_distance(benign_hue, malignant_hue)
value_gap = abs(benign_v - malignant_v)
print(f"hue gap {hue_gap:.2f} degrees, brightness gap {value_gap:.2f}")
hue_rows, bright_rows = jitter_changes(
    benign_paths,
    hue_levels=(0.02, 0.05, 0.1, 0.5),
    brightness_levels=(0.1, 0.3, 0.6),
)
print("hue jitter (True means the shift is larger than the class gap)")
print(pd.DataFrame(hue_rows).assign(exceeds=lambda frame: frame["mean_change_deg"] > hue_gap))
print("brightness jitter")
print(pd.DataFrame(bright_rows).assign(exceeds=lambda frame: frame["mean_change_v"] >
value_gap))

# %% [markdown]
# Horizontal flip and a small rotation do not change the diagnosis. Every tested hue
# setting, including 0.02, moves hue by more than the 4.6 degree gap between benign and
# malignant lesions, so hue jitter is not used. Brightness 0.1 stays under the brightness
# gap; 0.3 and 0.6 do not. A clinician should accept this list before training: flip,
# a 15 degree rotation, and brightness/contrast of 0.1.

# %% [markdown]
# ## A3.6 Lesion-level dataset

# %%
transform = transforms.Compose(
    [transforms.Resize((224, 224)), transforms.ToTensor()]
)
label_map = {name: i for i, name in enumerate(sorted(lesions["dx"].unique()))}
dataset = LesionDataset(
    lesions.iloc[:800], IMG_DIR, "dx", transform, label_map=label_map
)
loader = __import__("torch").utils.data.DataLoader(dataset, batch_size=8)
batch = next(iter(loader))
assert batch["derm"].shape == (8, 3, 224, 224)
assert batch["clinical"].shape == (8, 3, 224, 224)
expected = lesions.iloc[:8]["lesion_id"].tolist()
assert list(batch["lesion_id"]) == expected

seen = []
for batch in loader:
    seen.extend(batch["label"].tolist())
expected_counts = lesions.iloc[:800]["dx"].map(label_map).value_counts().sort_index()
got_counts = pd.Series(seen).value_counts().sort_index()
assert expected_counts.to_dict() == got_counts.to_dict()

```

```

synthetic = images.iloc[:4][["lesion_id"]].copy()
probs = np.array(
    [
        [1.0, 0.0, 0.0],
        [0.0, 0.0, 1.0],
        [0.0, 1.0, 0.0],
        [0.0, 1.0, 0.0],
    ]
)
# first two rows share a lesion in this table only if we force it
synthetic.loc[:, "lesion_id"] = ["L1", "L1", "L2", "L2"]
aggregated = aggregate_predictions(probs, synthetic)
assert aggregated.loc[aggregated["lesion_id"].eq("L1"), "prediction"].item() == 0
assert aggregated.loc[aggregated["lesion_id"].eq("L2"), "prediction"].item() == 1
print(aggregated)

# %% [markdown]
# The unit of evaluation is the lesion, not the photograph. The two views are one disease
# and one label. Scoring both photographs treats one patient as two cases, and if one view
# was in the training set the image score measures memory of that lesion.

```